

План оптимізації та впровадження ентропійного стиснення (Phase 2)

1. Глибокий профілінг та аудит (Nsight Systems / Compute)

Завдання: Аналіз поточної продуктивності та очищення коду.

- **Timeline Analysis:** Виявлення розривів у роботі GPU, спричинених синхронним копіюванням даних.
 - **Code Cleanup:** Видалення застарілого або дубльованого коду IDCT у CUDA-кernels для вивільнення регістрів та підвищення **Occupancy**.
 - **Baseline:** Фіксація поточного FPS та часу обробки одного кадру для порівняння з оптимізованою версією.
-

2. Оптимізація вхідних даних: Перехід на `uint8_t`

Завдання: Зменшити об'єм даних, що передаються на GPU, у 4 рази (з 32-бітних `float` до 8-бітних `uint8_t`).

- **Upload Optimization:** Зміна формату вхідних планарних буферів. Це критично знизить навантаження на шину PCIe.
 - **Kernel Update:** Адаптація DCT-кernels для зчитування `uint8_t`. Конвертація у `float` (або `fixed-point`) відбуватиметься вже всередині ядра безпосередньо перед обчисленнями.
 - **Memory Layout:** Оновлення структури `Frame` для підтримки компактного зберігання.
-

3. Ентропійне стиснення (CPU Reference)

Завдання: Реалізація еталонних алгоритмів на CPU для оцінки потенційного **Compression Rate**.

- **Варіанти:**
 1. **RLE (Run-Length Encoding):** Стиснення серій нулів після квантування.
 2. **Huffman:** Кодування на основі частот символів.

- 3. **RLE + Huffman:** Комбінований підхід для оцінки максимального стиснення.
 - 4. **RLE + Arithmetic**
 - **Аналіз:** Порівняння коефіцієнта стиснення для різних типів даних (колір, нормалі, глибина).
-

4. Масштабування блоків (8x8, 16x16, 32x32)

Завдання: Дослідження впливу розміру блоку на якість та швидкість обробки.

- **Імплементація:** Написання варіацій DCT-ядер для різних розмірів блоків.
 - **Аналіз ресурсів:** Оцінка споживання **Shared Memory**. Блоки 32x32 можуть дати краще стиснення на плавних градієнтах, але суттєво обмежити кількість активних варпів на мультипроцесорі (SM).
-

5. Імплементація Huffman на GPU (Бітовий потік)

Завдання: Розробка паралельного алгоритму запису стиснених даних змінної довжини.

- **Крок А: Підрахунок довжин.** Кожен блок обробляється окремо, і результат його стиснення (кількість бітів) записується у масив довжин.
 - **Крок Б: Розрахунок офсетів.** Використання **Prefix Sum (Scan)** на масиві довжин для отримання точних позицій у фінальному бінарному файлі для кожного блоку.
 - **Крок В: Паралельний запис.** Потoki записують біти у виділені сегменти глобальної пам'яті.
 - **Формат бінарника:** Збереження масиву довжин блоків у заголовок файлу. Це дозволить декомпресору миттєво розрахувати офсети для паралельного зчитування.
-

6. Виправлення CUDA Streams: Pinned Memory

Завдання: Забезпечення справжнього асинхронного виконання через заміну Pageable memory на Pinned memory.

- **Pinned Memory (`cudaHostAlloc`):** Переведення всіх буферів завантаження на "закріплену" пам'ять, що дозволяє драйверу використовувати DMA (Direct Memory Access).

- **Asynchronous Concurrency:** Виправлення логіки `cudaMemcpyAsync`, щоб копіювання наступного кадру відбувалося одночасно з обчисленнями поточного кадру в іншому стрімі.
-

7. Паралелізація конвеєра (Multithreading)

Завдання: Усунення простоя GPU через повільне читання з диска та парсинг зображень.

- **Thread-safe Queue:** Впровадження черги завдань типу Producer-Consumer.
 - **Розподіл потоків:**
 1. **I/O Thread:** Асинхронне читання сирих файлів з диска.
 2. **Parsing Threads:** Паралельний парсинг форматів (PNG/EXR) та перетворення в планарний `uint8_t` формат.
 3. **Worker/Main Thread:** Керування CUDA-стрімами та відправка підготовлених даних на GPU.
 - **Мета:** Повністю приховати час підготовки даних за часом виконання обчислень на відеокарті.
-

Очікуваний результат:

Система, що працює в повному асинхронному режимі, де завантаження даних (у компактному `uint8_t` форматі) та їх стиснення відбуваються паралельно. Наявність паралельного Хаффмана на GPU з масивом довжин блоків забезпечить швидкий доступ до даних при декомпресії.